Foldable & Traverse

Season: 6 · FP Foundations · Category: Functional · Level: Intermediate

Intent

- Generic folds and effectful walks over structures.

Problem

- Without this pattern, code tends toward: New things up with conditionals and duplicated construction.

Real-world Analogy

- Vacuuming every room (fold) vs inspecting each with a checklist (traverse).

When to Use

- Pure functional style in Java 25 + Zazr.
- The design force named in Intent shows up more than once.

When Not to Use

- Throwing exceptions everywhere is 'fine' for a script.
- A simpler language feature (record, sealed switch, function value) already solves it.

Pattern Structure

- See mermaid diagram in explanation.md

Classic Implementation

```java
package com.buildwithwaqar.patterns.functional.foldable;

import com.buildwithwaqar.patterns.functional.monoid.Monoid;
import com.buildwithwaqar.patterns.functional.typeclass.Applicative;
import com.buildwithwaqar.patterns.functional.typeclass.Kind;

import java.util.ArrayList;
import java.util.List;
import java.util.function.BiFunction;
import java.util.function.Function;

/** Foldable + Traverse for List — educational encodings. */
public final class Foldable {
    private Foldable() {}

    public static <A, B> B foldLeft(List<A> xs, B z, BiFunction<B, A, B> f) {
        B acc = z;
        for (A x : xs) acc = f.apply(acc, x);
        return acc;
    }

    public static <A> A fold(List<A> xs, Monoid<A> m) {
        return foldLeft(xs, m.empty(), m::combine);
    }

    public static <F, A, B> Kind<F, List<B>> traverse(Applicative<F> A, List<A> xs, Function<A, Kind<F, B>> f) {
        Kind<F, List<B>> acc = A.pure(new ArrayList<>());
        for (A x : xs) {
            Kind<F, B> fb = f.apply(x);
            acc = A.ap(A.map(acc, list -> (b -> { list.add(b); return list; })), fb);
        }
```

Classic Implementation (2)

```
        return acc;
    }
}
```

Modern Java 25 / Functional

- Primary examples use Zazr; VAVR shown in comparison episode.

Worked Example

```
See the JUnit test beside the implementation.
Run: mvn -Dtest=*Test test
Focus package matching: functional/foldable/
```

Pitfalls

- Zazr rejects null inside Option/collections.
- Keep interpreters at the edge.

Interview Q&A

- Q: Interview: what problem does Foldable & Traverse solve? A: Generic folds and effectful walks over structures.
- Q: How would you test this? A: Prefer injecting interfaces or interpreters; keep demos deterministic (fixed clocks, canned inputs).

Key Takeaways

- Generic folds and effectful walks over structures.
- Prefer Java 25 sealed types / records / virtual threads where they simplify the classic shape.
- Ship working tests with every episode.

Video Script Highlights

- [0:00] Hook — Open with the analogy: Vacuuming every room (fold) vs inspecting each with a checklist (traverse). [0:40] Intent — State th...